

Chapter 26: Kotlin Multiplatform

By Victoria Gonda

As of 2019, Kotlin is the preferred language to use for Android development. On iOS, the Swift language has replaced Objective-C as the de facto development language.

Swift and Kotlin have a multitude of similarities, including static typing, type safety, support for the functional and OOP paradigms, and safe-handling of null values. The languages are also syntactically very similar.

The advent of Kotlin/Native has opened up the possibility of integrating Kotlin code into your iOS projects. In fact, the **Kotlin Multiplatform (KMP)** approach is beginning to take off as a cross-platform toolset for iOS, Android, and beyond.

In this chapter, you'll use **Android Studio** and **Xcode** to create a KMP project.

Note: Because KMP can also be used with JavaScript, in documentation you might also see reference to **KMM** which is Kotlin Multiplatform Mobile, KPM specifically used for Android and iOS.

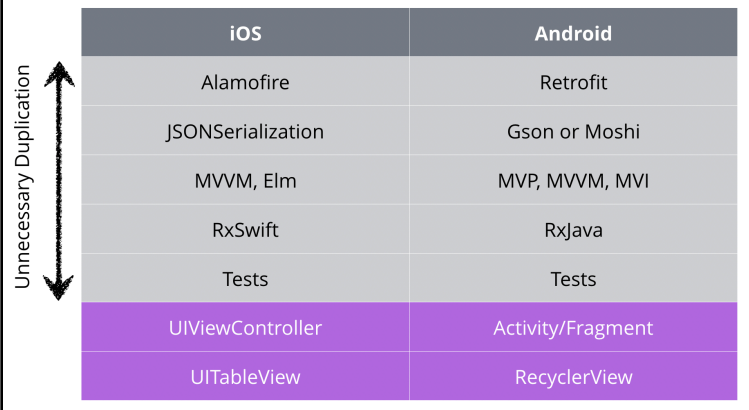
The KMP Approach

Typical apps on iOS and Android pull down data from the Internet, parse the data into objects within the app code, and cache some of the network data locally in a database.

For iOS, you might use a library like **Alamofire** for networking and something like **JSONSerialization** to parse the data received from the network into objects. You'd use **Core Data** to store the data locally in a database.

You'd implement an architecture pattern like **MVVM** to structure your app code. You might use a library like **RxSwift** to make your code more declarative. And you'd have tests for the view models and other parts of the app. You'd show lists of data in a **UITableView** in a **UIViewController**.

On Android, you'd have analogs of all of that code. You might use **Retrofit** for networking, **Gson** or **Moshi** for parsing the JSON, **Room** for storing the data in a database, a pattern like **MVP** or **MVVM** for the architecture, and maybe use **RxJava** in the app. You'd repeat similar tests. And you'd show list data in a **RecyclerView** in an **Activity** or **Fragment** using an adapter.



iOS	Android
Alamofire	Retrofit
JSONSerialization	Gson or Moshi
MVVM, Elm	MVP, MVVM, MVI
RxSwift	RxJava
Tests	Tests
UIViewController	Activity/Fragment
UITableView	RecyclerView

That's a lot of duplication even for a simple app. Imagine that there were numerous screens in your app, with more data, networking calls, and local caching of the remote data, as there would be in a full-featured app. The amount of code duplication would grow essentially linearly with the size of the app, as would the amount of time and effort to first produce and then maintain the two apps for the two platforms.

Other Cross-Platform Frameworks

Reducing this duplication in targeting both iOS and Android has long been a vision of many developers and organizations in the mobile world. Early attempts included web frameworks such as **PhoneGap**.

Organizations like Microsoft have produced tools like **Xamarin**, which uses C# and .NET to target iOS and Android. **React Native**, a derivative of the **React** web framework from Facebook, has become a popular modern framework for mobile. Most recently, Google has released the cross-platform framework **Flutter**, which uses its own runtime to allow apps written in **Dart** to perform at native speeds on iOS and Android.

These and other cross-platform toolkits have had great promise, but none have truly taken hold in the mobile development world. There are many reasons for this, some technical, others, less technical. Just a few of the reasons are poor performance of the resulting apps, inconsistencies with the native user interfaces, an inability to stay up-to-date with the latest iOS and Android features, and developer loyalty to and expertise with the native SDKs.

This is where Kotlin Multiplatform comes in. It's not a cross-platform framework. In fact, it's not a framework at all. It's more of an approach to mobile app development that in some ways gives you the best of all possible worlds.

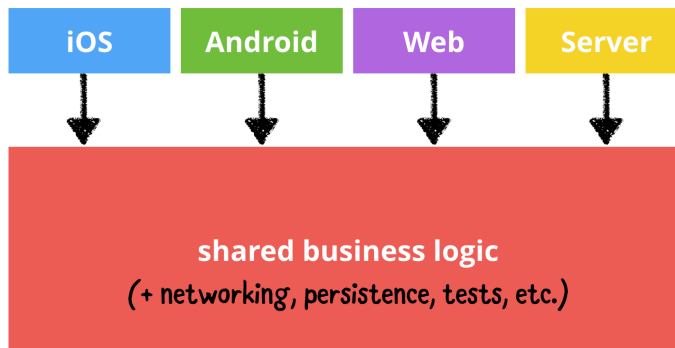
Kotlin Multiplatform has a number of distinct advantages over the other approaches:

- Android developers can leverage their Kotlin skills within shared code used by both the iOS and Android apps.
- iOS developers can use their knowledge of Swift to quickly get up to speed with Kotlin and contribute to the shared Kotlin code.
- The Android UI code remains Kotlin, and the iOS UI code remains in Swift, so the user interfaces can take advantage of the latest improvements on both platforms.
- Performance of both the iOS and Android apps matches the performance of purely platform-specific native apps.

Like the other approaches to cross-platform, Kotlin Multiplatform promises to cut down on the time and effort required to produce apps for both iOS and Android.

Sharing Code

With Kotlin Multiplatform, you reduce code duplication by putting code common to all front-end apps into one module or shared project. This includes business logic, and things like networking code, data parsing, data persistence, and more.



You can use various architectural patterns, and in a large app, you might consider something like **Clean Architecture**, where all the inner layers of the software are shared between front-ends, and only the outermost layer is unique to a given platform such as iOS, Android, Web, or Server. This significantly reduces the amount of duplication in the software, as most or all of the logic and functionality is only written in one place.

Another benefit of KMP, especially on a larger app development project, is that you can divide your team up into groups that work in different areas. You can have a group dedicated to the shared code, a group dedicated to the Android user interface, and a group dedicated to the iOS user interface. Each of these groups can have subgroups for a larger app.

An additional possible benefit, if your team's expertise is favored towards Kotlin, is that you can even write the iOS user interface code in Kotlin instead of Swift. This is not recommended in general, as it goes somewhat against the grain of what can be achieved with Kotlin Multiplatform. But it may be a good approach for an independent Android developer looking to create an iOS version of their app.

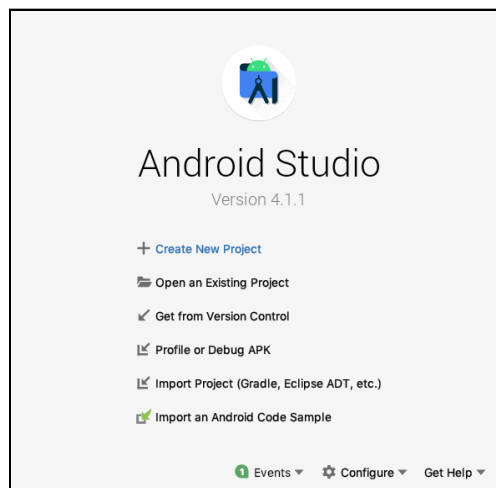
HelloKMP

You're going to build a simple app named **HelloKMP** that shares Kotlin code between iOS and Android apps. You'll start in Android Studio and first setup the Android app project and the project level build files for the entire KMP project.

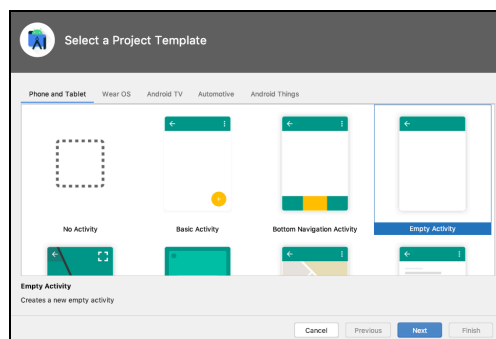
You'll want to use Android Studio 4.1 or later with SDK 30 or later installed and Kotlin plugin 1.4.20 in order to follow along.

Note: If you don't have Android Studio installed, you can get it at <https://developer.android.com/studio/>.

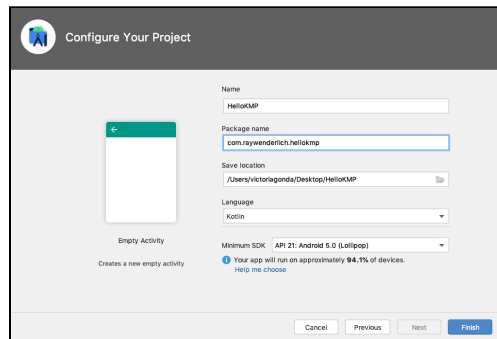
Open Android Studio and click the **Create New Project** link on the welcome screen.



Then choose **Empty Activity** and hit **Next**.

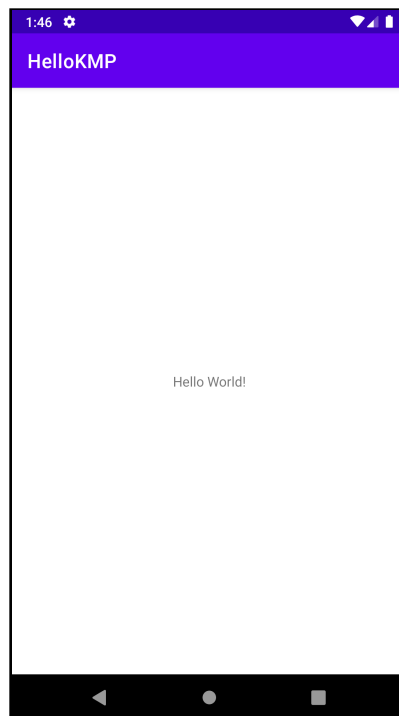


Name the project **HelloKMP**. The package name should be something like `com.raywenderlich.hellokmp`, and the language is Kotlin. Choose a location for the project and use API 21 for the minimum Android SDK level supported. Then click **Finish**.



The project will open and a **Gradle** build will run.

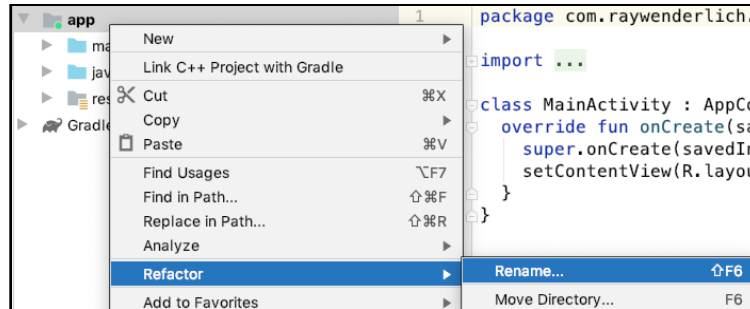
In an Android emulator, run the initial Android app to make sure it builds and runs.



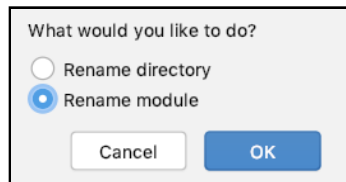
Renaming the app folder

The name of the android **app** folder is ideally something like **androidApp** instead of just the default **app**, in order to distinguish that part of the project as being the Android app.

To rename the folder, right click on the folder name and select **Refactor** ▶ **Rename**.



When prompted, select **Rename module**.



Then, enter **androidApp** as the name.

Finally, build and run the Android app in an emulator to make sure all is good after these changes.

Shared project

You're going to build up the shared project more or less by hand. That way, you'll see all that goes into creating the shared project.

In a terminal window and from the root of the HelloKMP project, first make directories for the shared project, using the `-p` option which creates parent directories as needed:

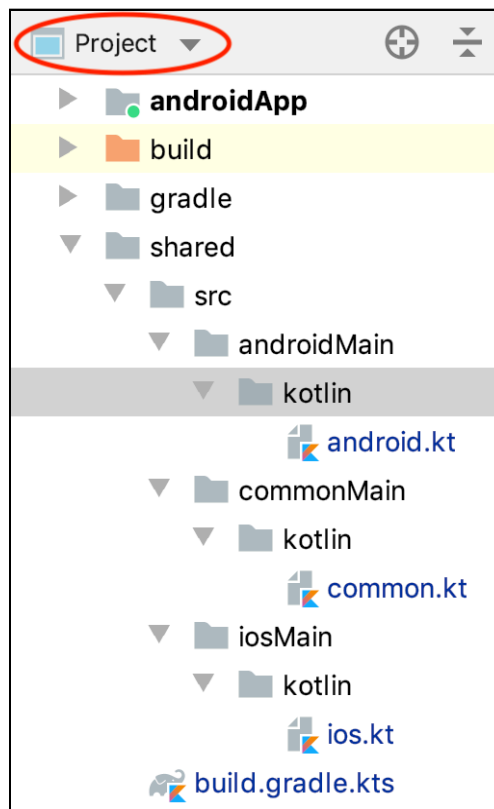
```
mkdir -p shared/src/androidMain/kotlin
mkdir -p shared/src/commonMain/kotlin
mkdir -p shared/src/iosMain/kotlin
```

You've made **commonMain**, **androidMain**, and **iosMain** folders.

Next, use the `touch` command to add the Kotlin source files you'll start with, along with a **build.gradle.kts** file for the shared project:

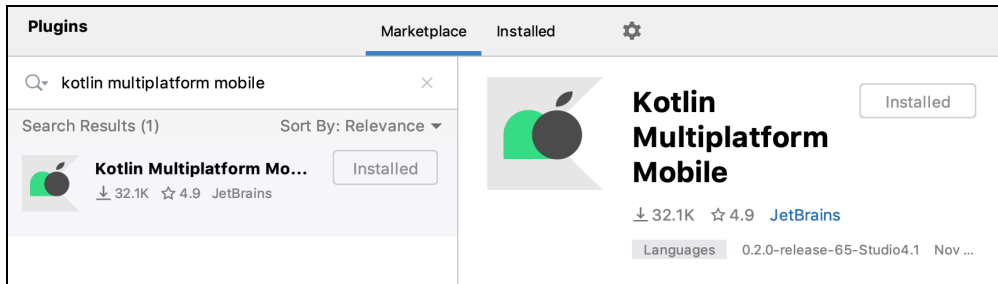
```
touch shared/src/commonMain/kotlin/common.kt
touch shared/src/androidMain/kotlin/android.kt
touch shared/src/iosMain/kotlin/ios.kt
touch shared/build.gradle.kts
```

Back in Android Studio, switch the **Project** panel to **Project** view to see the folder and file structure of the project, including the folders and files you just added.



Configuring your environment

To work with Kotlin Multiplatform projects in Android Studio, you need to download the Kotlin Multiplatform Mobile plugin. Open **Preferences** ▶ **Plugins** ▶ **Marketplace** and search for **Kotlin Multiplatform Mobile**.



Install the plugin and restart Android Studio.

Note: There are generators to create Kotlin Multiplatform modules and apps included in this plugin. This chapter is not using them so you can learn the individual parts.

Shared code build file

The shared code will be turned into the a **jar** file for running with the Android app, and an iOS **framework** for running with the iOS app. The shared project **build.gradle.kts** file is a Kotlin file where you will specify how that is done.

First, update the **settings.gradle** file at the project root to include building the shared code. Before the include for **androidApp**, add:

```
include ':shared'
```

Next, you turn to writing the build file for the shared project, **shared/build.gradle.kts**.

First, set up the **multiplatform** plugin in order to pull in the compilers you need, such as the Kotlin/Native compiler.

```
plugins {  
    kotlin("multiplatform")  
    id("com.android.library")  
}
```

If you see a popup indicating that there is a new build context, be sure to accept the proposed change.

Add a `kotlin` section to define your targets:

```
kotlin {  
  
}
```

Inside the `kotlin` section, first define some targets:

```
android()  
ios {  
    binaries {  
        framework {  
            baseName = "shared"  
        }  
    }  
}
```

This means you're targeting both iOS and Android.

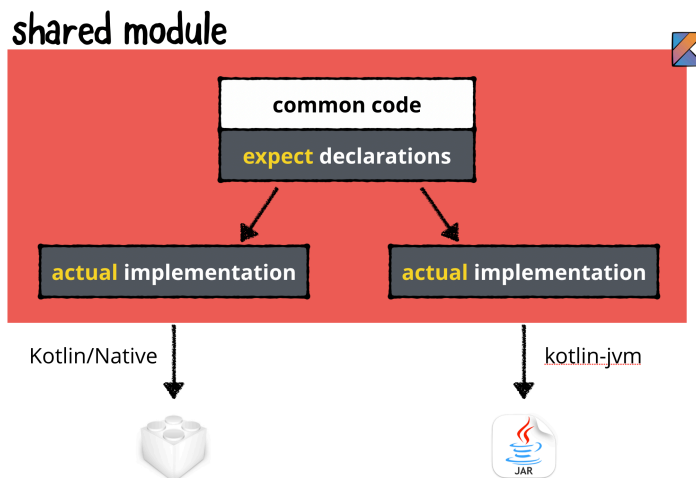
Then, add `sourceSets` sections to the `kotlin` section, in which you define dependencies for the shared code:

```
sourceSets {  
    val commonMain by getting  
    val androidMain by getting {  
        dependencies {  
            implementation("com.google.android.material:material:  
1.2.1")  
        }  
    }  
    val iosMain by getting  
}
```

Having made changes to Gradle files in the project, you can finish up by syncing the project files to make sure there are no errors. The first sync will take awhile, since Android Studio needs to pull down the Kotlin/Native compiler.

expect and actual

Compiling the entire shared project for all platforms is not the approach taken by Kotlin Multiplatform. Instead, a certain amount of code is common to all platforms, but some amount of the shared code is unique to each platform. Kotlin includes the **expect/actual** mechanism to allow for this.



You can think of **expect** as defining something like an **interface** in Kotlin or a **protocol** in Swift. You use **expect** to say that the shared common code *expects* something to be available in the compiled code for all platforms. You then use **actual** to give the *actual* version of that something for each separate platform.

In the usage of **expect** and **actual** for HelloKMP, you're going to expect that each platform can tell the shared code what its name is as a string using a `platformName()` function. You can then use that `platformName()` function in other parts of the shared code.

You can use `expect` on entities such as functions, classes, or properties. Like a canonical interface, items tagged with `expect` do not include implementation code. That's where `actual` comes in.

In **iOSMain** and **androidMain**, you need to provide the `actual` version of the items specified with `expect` in the common code. The **androidMain** code will be compiled using `kotlin-jvm`, and the **iOSMain** code will be compiled by Kotlin/Native. Each will be combined with the compiled version of the common code for the respective platforms.

In **shared/src/commonMain/kotlin/common.kt**, add the package and an `expect` for the `platformName()` function:

```
package com.raywenderlich

expect fun platformName(): String
```

You'll see an error saying that there are no `actual` implementations for either JVM, which means Android, or Native, which means iOS.

Next, add a `Greeting` class to the same file in which you use the result of calling `platformName()`:

```
class Greeting {
    fun greeting(): String = "Hello, ${platformName()}"
}
```

Since you don't use `expect` on this class, this is a Kotlin class that is the same for all platforms.

In **shared/src/androidMain/kotlin/android.kt**, add an `actual` version of `platformName` for Android.

```
package com.raywenderlich

actual fun platformName(): String {
    return "Android"
}
```

In **shared/src/iosMain/kotlin/ios.kt**, add an actual version of `platformName` for iOS.

```
package com.raywenderlich

import platform.UIKit.UIDevice

actual fun platformName(): String {
    return "${UIDevice.currentDevice.systemName()}"
}
```

Notice the platform package import of a **UIKit** class. There may be an unresolved reference error here. That's okay for now.

If you click on the **E** in the gutter of the Android Studio editor, you get taken to the corresponding expected definition. You see that the error you had before is gone now that you have actual versions of `platformName()` for both platforms.

Clicking the **A** in the gutter, you can choose to navigate to any of the actual implementations.

Building your projects

There are two small things you need to do before building:

1. Adding an Android manifest
2. Giving Gradle an SDK version for Android

First, right click on **shared/androidMain** and select **New File**. Name this file **AndroidManifest.xml**

Open the file and add these contents:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/
android"
    package="com.raywenderlich">

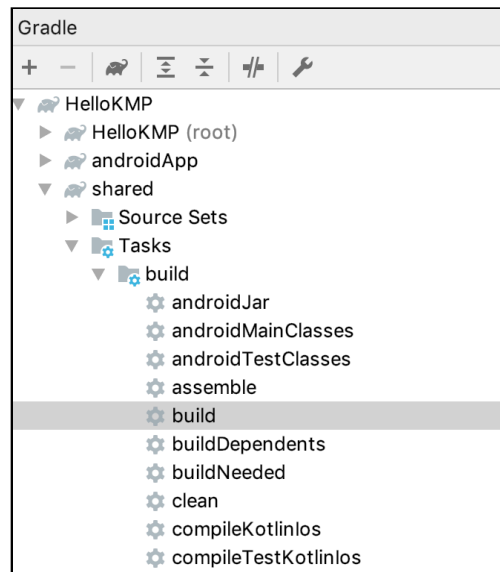
</manifest>
```

Then, Add this to the bottom of **shared/build.gradle.kts**:

```
android {  
    compileSdkVersion(29)  
    sourceSets["main"].manifest.srcFile("src/androidMain/  
AndroidManifest.xml")  
    defaultConfig {  
        minSdkVersion(21)  
        targetSdkVersion(29)  
    }  
}
```

Now you have shared code that you can build.

Open the **Gradle** panel, find the **build** task folder under **shared/Tasks**, then double-click on **build**.



You can then watch the code build in the **Build** panel. You can see Gradle going through a number of build stages and tasks. This will typically take a bit of time to run, especially after a clean of the shared project.

You'll see a **BUILD SUCCESSFUL** message when it's done.

So now you've successfully built the shared project, in which you've defined a Greeting class that shows a greeting that's customized for the platform that you're running the app on.

Shared code from Android

Now it's time to use the shared library from the Android app.

In the **androidApp/build.gradle** file, add a `packagingOptions` call within the `android` block:

```
packagingOptions {  
    exclude 'META-INF/*.kotlin_module'  
}
```

This addresses a build error that might occur due to duplicated files in the build.

Then add a dependency of the Android project on the shared project to the `dependencies` block.

```
implementation fileTree(dir: 'libs', include: ['*.jar'])  
implementation project(':shared')
```

Sync the project Gradle files before you continue.

In **androidApp/src/main/res/layout/activity_main.xml**, add an id of greeting on the `TextView` that is included in the template android project:

```
android:id="@+id/greeting"
```

Then in **androidApp/src/main/java/com.raywenderlich.hellokmp/MainActivity.kt** in the `onCreate()` function, set the text on the greeting `TextView` by calling into the shared code and using the `Greeting` class:

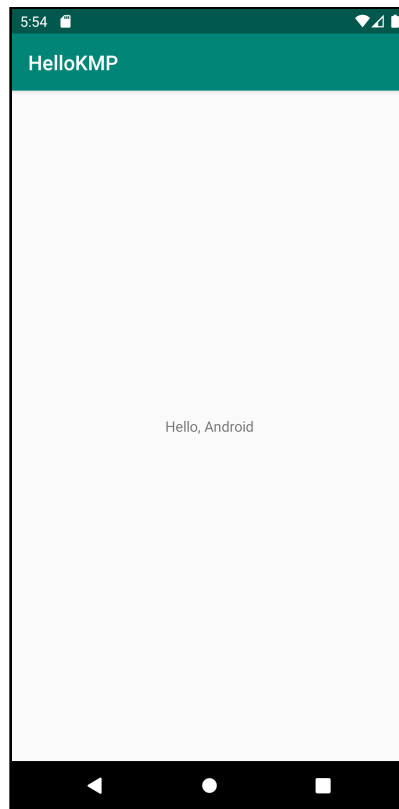
```
findViewById<TextView>(R.id.greeting).text =  
    Greeting().greeting()
```

You created a `Greeting` object and called its `greeting()` method.

You should see an import for `Greeting` pulled in when you use the **option+return** keystroke, along with an import for `TextView`:

```
import android.widget.TextView  
import com.raywenderlich.Greeting
```

Build and run the Android app.



There is your greeting that displays “Hello, Android” as determined by the shared code.

The iOS app

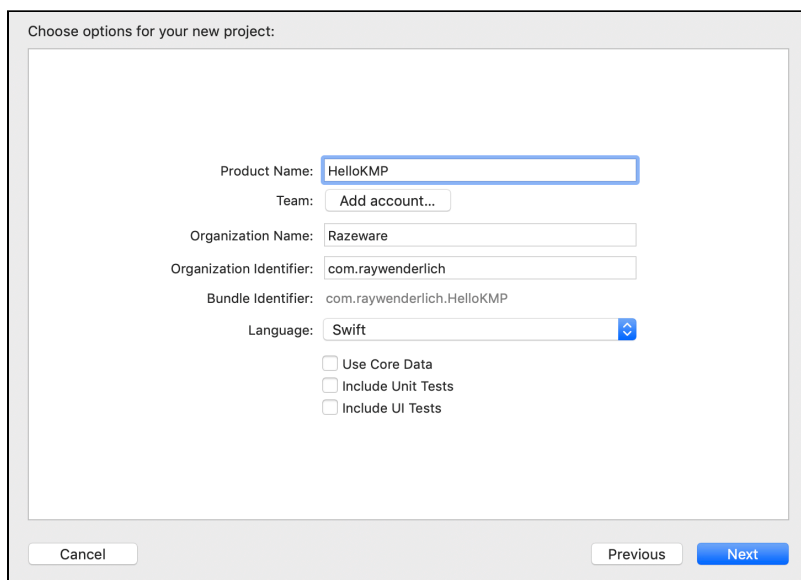
Having used the shared project in an Android app, you now turn to using the shared code in an iOS app. But first you need to setup the iOS app project itself.

In a terminal window at the HelloKMP project root, create a directory for the iOS app:

```
mkdir iosApp
```


Then switch to Xcode version 11.3 or later, and choose **File ▶ New ▶ Project**, pick **Single View App**, and click **Next**.

The product name is **HelloKMP**, the organization identifier is `com.raywenderlich`, and make sure the language is Swift:



Click **Next**, and place the project in the new **iosApp** folder you just made.

Now build and run the app in the iOS Simulator just to make sure it builds correctly.

Packing the iOS framework

Next, back in Android Studio, you need to add a task to the Gradle build file **shared/build.gradle.kts** for the shared project that will package the framework for Xcode.

Add this to the bottom of the file:

```
val packForXcode by tasks.create {
```

Inside this block is where you'll build your task.

Start by adding the following:

```
group = "build"
/// selecting the right configuration for the iOS
/// framework depending on the environment
```

```

/// variables set by Xcode build
val mode = System.getenv("CONFIGURATION") ?: "DEBUG"
val sdkName = System.getenv("SDK_NAME") ?: "iphonesimulator"
val targetName = "ios" + if (sdkName.startsWith("iphoneos"))
    "Arm64" else "X64"
val framework = kotlin.targets
    .getByName<KotlinNativeTarget>(targetName)
    .binaries
    .getFramework(mode)
inputs.property("mode", mode)
dependsOn(framework.linkTask)
}

```

This first section of the task sets a directory for the framework and determines the correct framework to build based on the selected target in the Xcode project, with a default of **DEBUG**.

Next, add this code to the bottom of the same block:

```

val targetDir = File(buildDir, "xcode-frameworks")
from({ framework.outputDirectory })
into(targetDir)

```

This section copies the file from the **build** directory into the **framework** directory:

Finally, add this to the block:

```

/// generate a helpful ./gradlew wrapper with embedded Java path
doLast {
    val gradlew = File(targetDir, "gradlew")
    gradlew.writeText("#!/bin/bash\n"
        + "export 'JAVA_HOME=$"
        + {System.getProperty("java.home")}'\n"
        + "cd '${rootProject.rootDir}'\n"
        + "./gradlew %*\n")
    gradlew.setExecutable(true)
}

```

Here, a bash script named **gradlew** is created in the framework directory that Xcode will call to build the shared framework.

The script uses the version of the JDK that is embedded in Android Studio.

At the bottom of the **build.gradle.kts** file, you need to specify that the shared code build task depends on the new **packForXcode** task:

```

tasks.getByName("build").dependsOn(packForXcode)

```

You then need to run a project Gradle sync due to the changes to **build.gradle.kts**

Now you can build the shared code into an iOS framework, using the Gradle panel in Android Studio like you did before.

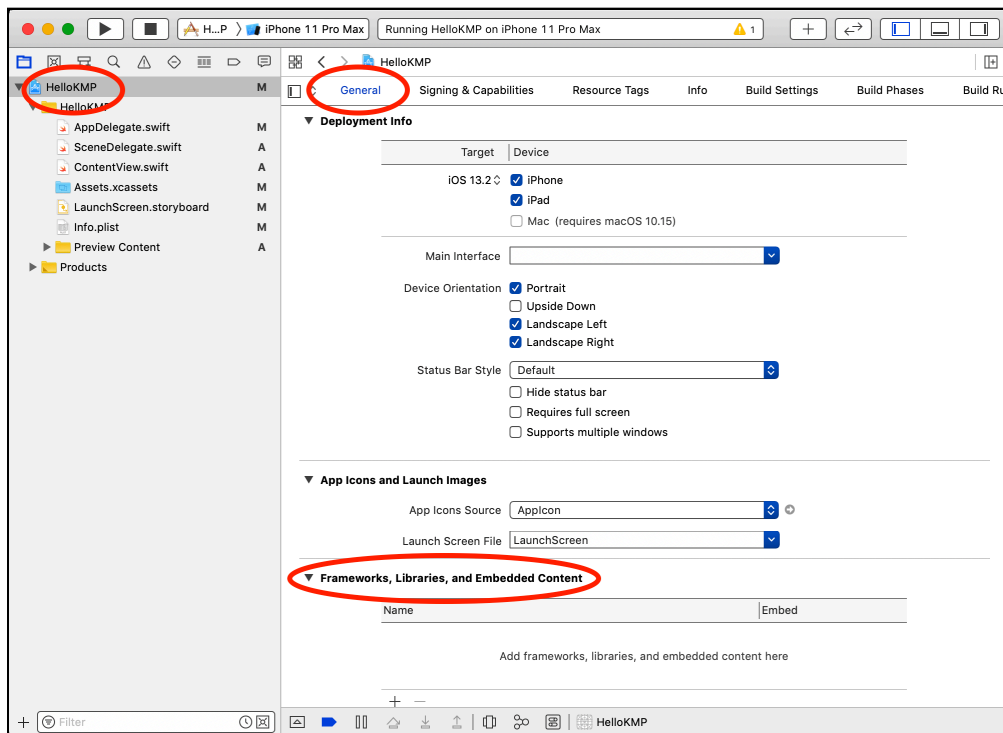
When the build is done, check that the packaged Xcode framework is in the expected directory.

```
ls -l shared/build/xcode-frameworks
```

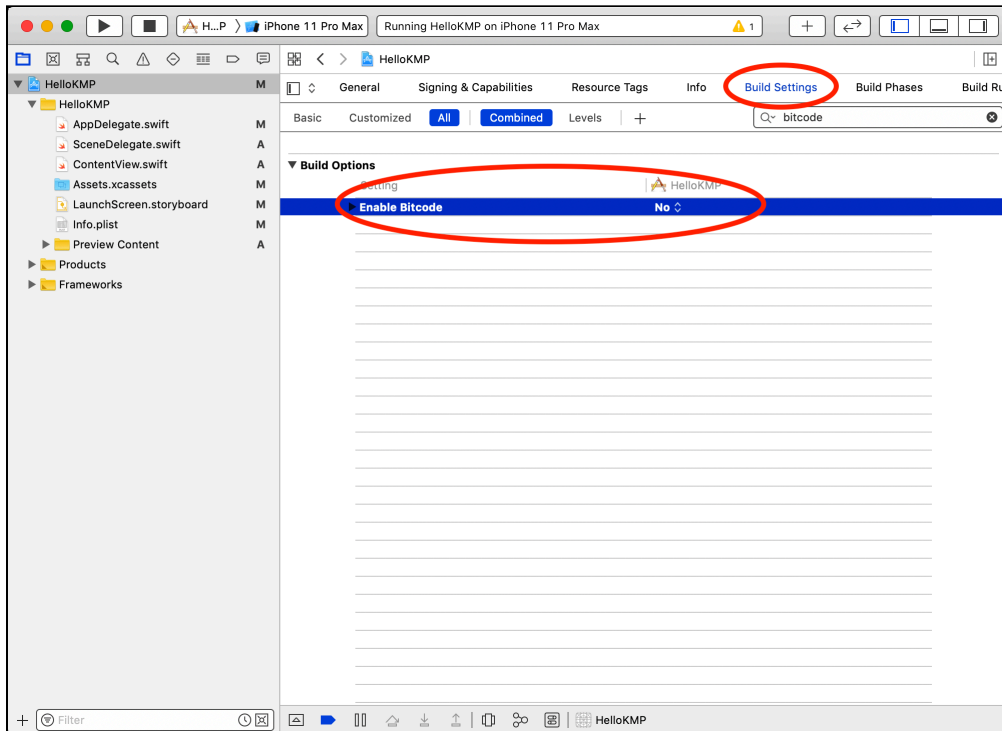
You should see the file **shared.framework** in the folder.

Go back to Xcode to finish the iOS app setup. Choose the app target and go to **General** settings.

In the **Embedded Binaries** section, click the plus and then select **Add Other** and **Add Files**. Navigate to and choose the shared framework **shared/build/xcode-frameworks/shared.framework**.



Since Kotlin/Native produces full native binaries, you need to disable the **Bitcode** feature for the project in **Build Settings**. Search on bitcode in the search box and choose **No** for **Enable Bitcode**.



Now you need to update framework search paths for the iOS project.

In **Build Settings**, search for **Framework Search Paths**, and then add the framework directory you setup in the new Gradle task, using `$(SRCROOT)/../shared/build/xcode-frameworks`.



Xcode will then set the absolute path based on the **SRCROOT** value.

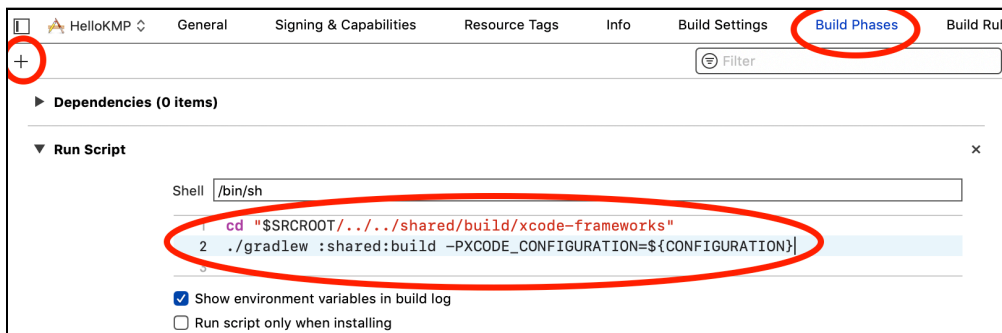
The last step you need for setting up the Xcode project is to add a new build phase to have Xcode build the shared code.

Switch to **Build Phases** and add a new **Run Script**.

In the run script, change the directory to the framework directory, and then call the bash script you created in the `packForXcode` task, passing in the Xcode configuration value.

```
cd "$SRCROOT/../../shared/build/xcode-frameworks"  
./gradlew :shared:build -PXCOD_CONFIGURATION=${CONFIGURATION}
```

Then move the new run script to the top of the Build Phases, just below Dependencies.

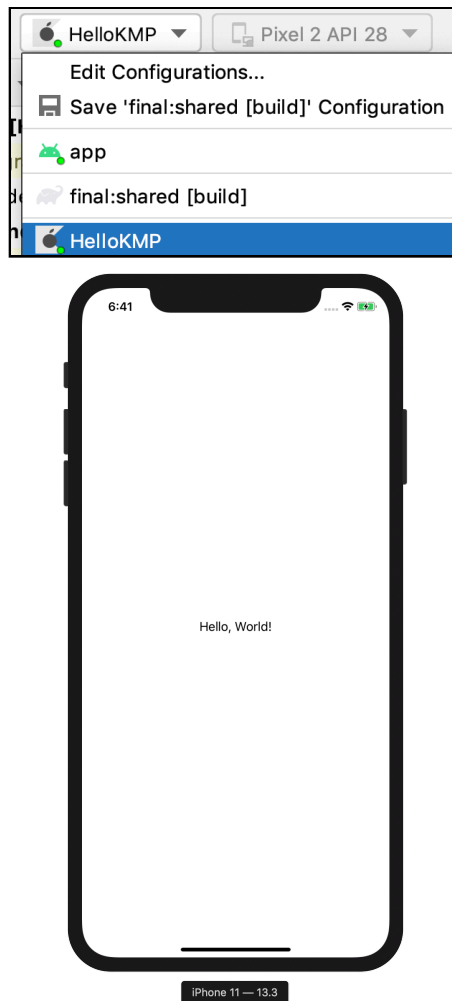


Now build and run the app to make sure there are no build errors due to any of these changes.

So you can run the iOS app from Android Studio, add the following line to **gradle.properties**:

```
xcodeproj=iosApp/HelloKMP
```

Sync Gradle. Then, select the iOS app to run.



Note: To run this project in Xcode, you need to do additional configuration to inform Xcode how to build using the shared Kotlin code.

Shared code from iOS

With the iOS app project in place and linked up with the shared project, next you'll use the shared code in the iOS version of the HelloKMP app. The iOS app will consist primarily of user interface code, relying on the shared code to do most of the work.

Open `iosApp/HelloKMP/HelloKMP/ContentView.swift` and add an import for the `shared` code to the top of `ViewController.swift`:

```
import shared
```

This gives you access to the shared code in the Swift file. It might give you an error in Xcode without the extra Xcode configuration, and that's okay.

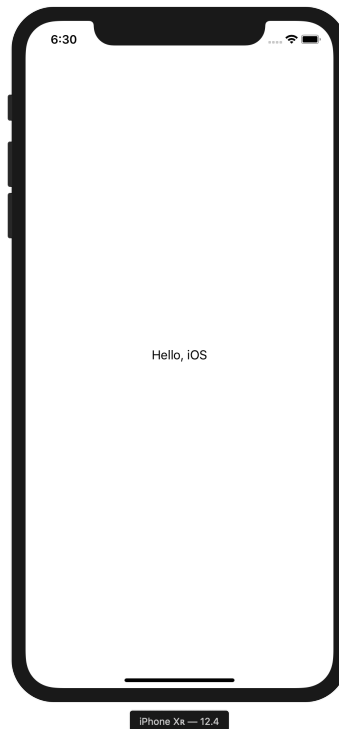
In the `ContentView` body, update the `Text` to call into the shared code:

```
Text(Greeting().greeting())
```

This simple Swift code is literally identical to the line in Kotlin that was used in the Android app. You create a `Greeting` object and then call its `greeting()` method.

You can now build and run the iOS app to see that everything is working.

When the app comes up in the simulator, there is your greeting that displays the iOS system name as determined in the shared code.



Challenge

You have a real albeit simple Kotlin Multiplatform app for iOS and Android that uses shared code between the two platforms. Your challenge for this chapter is to add the iOS `systemVersion` into the greeting in the iOS app. As a hint, you can use `UIDevice.currentDevice.systemVersion` to obtain the system version.

There are two things to consider when working on this challenge:

1. Where do you need to add this code to show the system version?
2. Do you need to rebuild the shared code before running the iOS app?

With those questions in mind, go ahead and tackle this challenge. You can check your solution in the challenge project for this chapter.

Key points

- **Kotlin Multiplatform** is a new and fast-growing approach to cross-platform app development.
- KMP lets you share Kotlin code between iOS, Android, web, server, and more.
- There are a number of advantages to KMP, including developer familiarity with Kotlin, native performance, native UI code, and the consolidation of your app business logic into a shared module across all platforms.
- You use the **expect** and **actual** keywords to create a common interface within the shared code that relies on concrete implementations on different platforms as needed.

Where to go from here?

You've just scratched the surface of Kotlin Multiplatform development in this chapter. There are a growing number of resources out there on KMP, so be sure to seek them out to see how to build more realistic apps, including doing things like networking, parsing JSON, and storing data locally in your app.

The Kotlin Multiplatform community is just getting started, so there's a great opportunity now to contribute to the KMP ecosystem.